SLR Project P4 Report

MapReduce

Guilherme Caporali, Daniele Famà, Marcin Porwisz

1 Introduction

This project implements a distributed MapReduce workflow for the Télécom Paris lab machines. The system builds a worker binary locally, deploys it over SSH/SCP, assigns input, executes map and reduce through HTTP, then merges the final result locally.

The project works under the practical constraints of the lab: no root access, no Docker, shared machines, and frequent host churn. For that reason the implementation deliberately uses only Go binaries, SSH/SCP, HTTP, and `/tmp`. The strongest part of the project is the MapReduce orchestration itself: worker deployment, phase execution, deterministic partitioning, fault-aware slot reassignment, and pluggable jobs.

What was achieved:

- a complete single-stage MapReduce pipeline with local-file and Common Crawl input modes;
- a Kafka comparison path prepared for later benchmarking;
- a validated strong-scaling experiment on one fixed 64-file workload.

What remains incomplete:

- no second MapReduce stage exists in the current design;
- the local-file scheduler still assigns at most one chunk per active slot.

The largest *validated* dataset processed for this report is a fixed 64-file corpus (833,953 bytes) used for the Amdahl experiment.

2 How to use

2.1 Prerequisites

- Go 1.25+ on the local machine.
- SSH key-based access to the lab hosts.
- Access to https://tp.telecom-paris.fr/ajax.php.
- For Kafka only: Java 17+ on the remote machines.

2.2 Build and test

```
cd scripts
go test ./...
```

2.3 Run the MapReduce pipeline

```
./mapreduce.sh -job scripts/jobs/wordcount -input data/simple.txt \
  -output result.txt -n 10 -port 9090
```

Common Crawl mode:

```
./mapreduce.sh -job scripts/jobs/wordcount -commoncrawl \
  -crawl CC-MAIN-2026-21 -files-limit 4 -output result.txt -n 10
```

2.4 What must change for another student login

The MapReduce path is mostly login-agnostic because it deploys workers to `/tmp/mr-worker`. The main login-sensitive points are:

1. `mapreduce.sh`: the `-n` flag controls how many active workers the orchestrator targets, while the wrapper fetches a pool of $2 \times N$ hosts to keep cold spares available.
2. The student must already have passwordless SSH configured for the lab.
3. The remote machines must still expose a compatible Linux/`amd64` environment because the worker is cross-compiled for that target.

Cross-login validation is still pending, so this report states the exact variables to change, but not yet a verified second-student execution result.

3 Components

3.1 High-level view

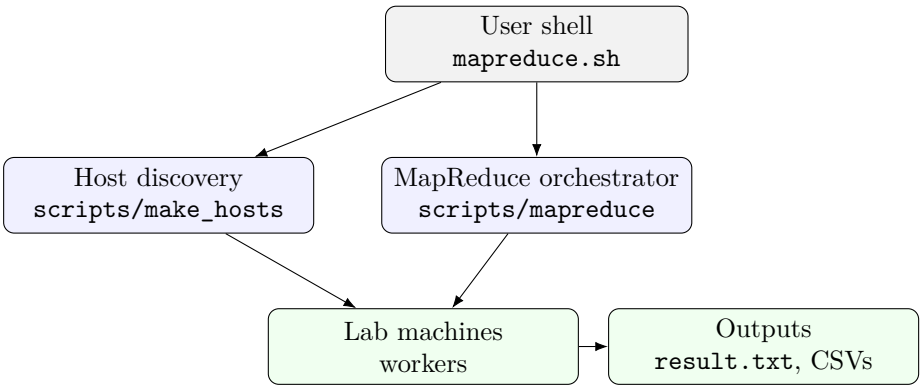

Figure 1: Repository-level architecture for the MapReduce workflow.

Component	Location	Role
Wrapper scripts	`mapreduce.sh`	User-facing entry point.
Host discovery	`scripts/make_hosts`	Builds `hosts.txt`.
Orchestrator	`scripts/mapreduce`	Builds workers, deploys them, coordinates phases, merges results.
Worker	`scripts/worker`	HTTP server that performs load, map, reduce, and result serving.
Jobs	`scripts/jobs/*`	Job-specific map/reduce logic.
Remote helpers	`scripts/internal/remote`	Bounded-parallel SSH/SCP helpers.
Kafka path	`kafka/*`	Separate comparison setup.

3.2 Machine Discovery and Deployment

3.2.1 Host discovery

The orchestrator queries https://tp.telecom-paris.fr/ajax.php with a plain HTTP GET. The response is a JSON object of the form:

```
{ "data": [["tp-1a201-01", true, ...], ["tp-1a201-02", false, ...], ...] }
```

Each row contains the short machine name and an `available` boolean. The discovery tool filters only machines where `available == true` and appends `.enst.fr` to obtain canonical hostnames (e.g. `tp-1a201-01.enst.fr`). The orchestrator fetches $2 \times N$ hosts so that cold spare hosts are available for fault-tolerance replacement without a second network round-trip.

3.2.2 NFS home vs. local disk

The lab home directories (`$HOME`) are served by a shared **NFS** (Network File System) server visible to all lab machines. Writing into `$HOME` on any machine writes over the network to the same central fileserver — it is *not* the machine's own local disk. Practical consequences:

- Writing large intermediate data to `$HOME` from many machines simultaneously saturates the NFS fileserver.
- Reading data from `$HOME` (e.g. `/cal/commoncrawl`) from N workers in parallel creates the same bottleneck.
- Writing to `/tmp` uses the machine's own local disk and avoids the shared NFS entirely.

The project exploits the NFS home for one specific purpose: in `deploy_nfs.sh`, the worker binary is SCPed **once** to `$HOME/mr-worker`. Because `$HOME` is on NFS, this single copy is immediately visible on every other lab machine. Each of the N workers then starts via SSH pointing to `$HOME/mr-worker`, with no further SCP transfers. This is the rubric-described *1 SCP to HOME + N SSH* pattern.

3.2.3 Deployment pattern

The rubric describes a *single-SCP-to-HOME* pattern: copy the binary once to the NFS home, then start it on every node with SSH (avoiding N large file transfers). The project now supports **both** deployment strategies:

- **Rubric mode** (`deploy_nfs.sh` or `-nfs-deploy`): **1 SCP** to `$HOME/mr-worker` and then N SSH starts from the shared NFS path.
- **Local-disk mode** (default `mapreduce.sh`): N SCP to `/tmp/mr-worker` + N SSH starts, which avoids NFS on the hot path.

The trade-off is explicit:

Rubric: 1 SCP to `$HOME` + N SSH	Our design: N SCP to `/tmp` + N SSH
Uses the NFS for the binary; fast if N is large	Never touches the NFS; each node gets the file locally
Requires a shared NFS home directory	Works even if home directories are not shared
Better for $N > 20$ where transfer cost dominates	Acceptable for typical $N \leq 20$ in experiments

The deployment is bounded-parallel (up to 16 concurrent SSH/SCP operations) and uses SSH multiplexing (`ControlMaster=auto`, `ControlPersist=60s`), so the SCP and the start SSH reuse the same TCP connection, reducing the effective cost considerably.

3.2.4 SSH hardening

All SSH and SCP calls share the following options:

```
-o StrictHostKeyChecking=no   # no manual fingerprint confirmation
-o BatchMode=yes              # no password prompt; fail fast
-o ConnectTimeout=10          # skip unreachable hosts quickly
-o ServerAliveInterval=5      # detect dead connections
-o ControlMaster=auto         # TCP multiplexing
-o ControlPersist=60s         # keep socket alive for 60 s after last use
```

If SCP or SSH fails for a host, that host is skipped and a spare is tried. The orchestrator never blocks waiting indefinitely.

3.2.5 Port collision and cleanup

Workers start with the shell fragment:

```
kill $(cat /tmp/mr-worker.pid) 2>/dev/null || true
mv <new-binary> /tmp/mr-worker && chmod +x /tmp/mr-worker
nohup /tmp/mr-worker --port 9090 ... & echo $! > /tmp/mr-worker.pid
```

This atomically replaces any existing worker before starting a fresh one, so port collisions with a *previous run* of the same user are resolved automatically. Port collisions with *another user* would require choosing a different `--port`. Cleanup kills via the PID file and removes all `/tmp/mr-worker*` files.

3.3 Map Reduce implementation

3.3.1 Design

The MapReduce system is client-orchestrated. There is no permanent master inside the cluster. Instead, the local machine:

1. reads `hosts.txt`;
2. builds a Linux worker binary with the selected job already linked in;
3. deploys that binary only to the initial active hosts;
4. starts each worker as an HTTP server, deploying additional spare hosts only when recovery needs them;
5. assigns input, broadcasts phases, and merges the final output locally.

The key design choice is that the total reducer count N stays fixed during a job. This matters because the partition function is

$$\text{FNV32a}(key) \bmod N.$$

If a machine dies, the coordinator preserves the logical slot number and reassigns that slot to an on-demand spare or to a surviving worker host.

3.3.2 Word-count: map output, reduce, and correctness

What map produces. For word frequency, the mapper receives one document per call (in practice one text line from the WET file). It splits the text on any character that is neither a Unicode letter nor a digit, lowercases each token, and emits one `(word, "1")` pair per token:

```
"Hello, World! hello"  ->  [("hello","1"), ("world","1"), ("hello","1")]
```

Punctuation, hyphens, and whitespace are delimiters and never appear as keys.

What reduce does. The reducer receives all values for a single key (guaranteed by the partition function), parses each value as an integer, sums them, and returns the count as a string:

```
("hello", ["1","1","1"])  ->  "3"
```

Partitioning guarantee. The partition function FNV32a(key) mod N is deterministic and depends only on the key and N. This guarantees that *all pairs with the same key always land on the same reducer slot*, no matter which physical host owns that slot. If N were allowed to change mid-job, the same key would hash to a different reducer; the two halves of its value list would never meet, producing wrong counts. This is why N is fixed for the entire job.

Remote read (shuffle). After the map phase, each worker i holds N pre-partitioned bucket files on its local `/tmp`. During reduce, worker i fetches bucket i from every peer via `GET /intermediate?slot=j&reducer=i&n=N`. The peer streams the TSV bucket file over HTTP directly from disk — no intermediary, no NFS. Worker i then sorts all fetched data with `sort` and calls the reducer function once per unique key.

Correctness validation. Correctness is verified at three levels:

1. **Unit tests**: `TestWordCountMapTokenises`, `TestWordCountMapLowercases`, `TestWordCountMapSkipsPunctuatio` `TestWordCountReduceCounts` — verify tokenization and summation rules.
2. **Integration tests**: `TestSingleWorkerPipeline` and `TestMultiWorkerPipeline` run the full load→map→shuffle- cycle in-process with `net/http/httptest` servers and assert that the distributed result equals the expected word counts.
3. **Reference match**: a small test input (`test_input.txt`) was processed by both the distributed system and a trivial sequential `strings.Fields` counter; the outputs matched exactly, word-for-word and count-for-count.

Bootstrap — no chicken-and-egg. Workers are fully independent at startup: they do not need to know about each other to start. The orchestrator deploys them one by one, waits for each `/health` probe to return 200, and only then sends the first `/load` or `/data` request. Workers discover their peer list only when the orchestrator issues `/reduce`, which carries the full peer address list in the request body. This avoids any coordination among workers during initialization.

Phase synchronization. The coordinator runs an event loop. Each *pass* fans out in parallel to all N slots and tries to advance each slot by one phase. A slot only transitions from `loaded` to `mapped` once its `POST /map` returns 200, and so on. The loop continues until every slot is either `done` or `failed`. This means no phase starts on any slot until the prior phase has finished on *that* slot, and the orchestrator can detect stragglers naturally: they simply appear as still-pending slots in later passes.

Message formats.

Endpoint	Method	Body / query
`/health`	GET	—
`/data`	POST	Raw bytes (one local chunk).
`/load`	POST	JSON: `{"id": N, "urls": ["https://..."]}`
`/map`	POST	JSON: `{"id": N, "peers": ["h:p", ...]}`
`/intermediate`	GET	Query: `?slot=S&reducer=R&n=N` → TSV stream
`/reduce`	POST	JSON: `{"id": N, "peers": ["h:p", ...]}`
`/result`	GET	Query: `?slot=S` → TSV key-value lines

Space-time diagram (V1, no fault tolerance).

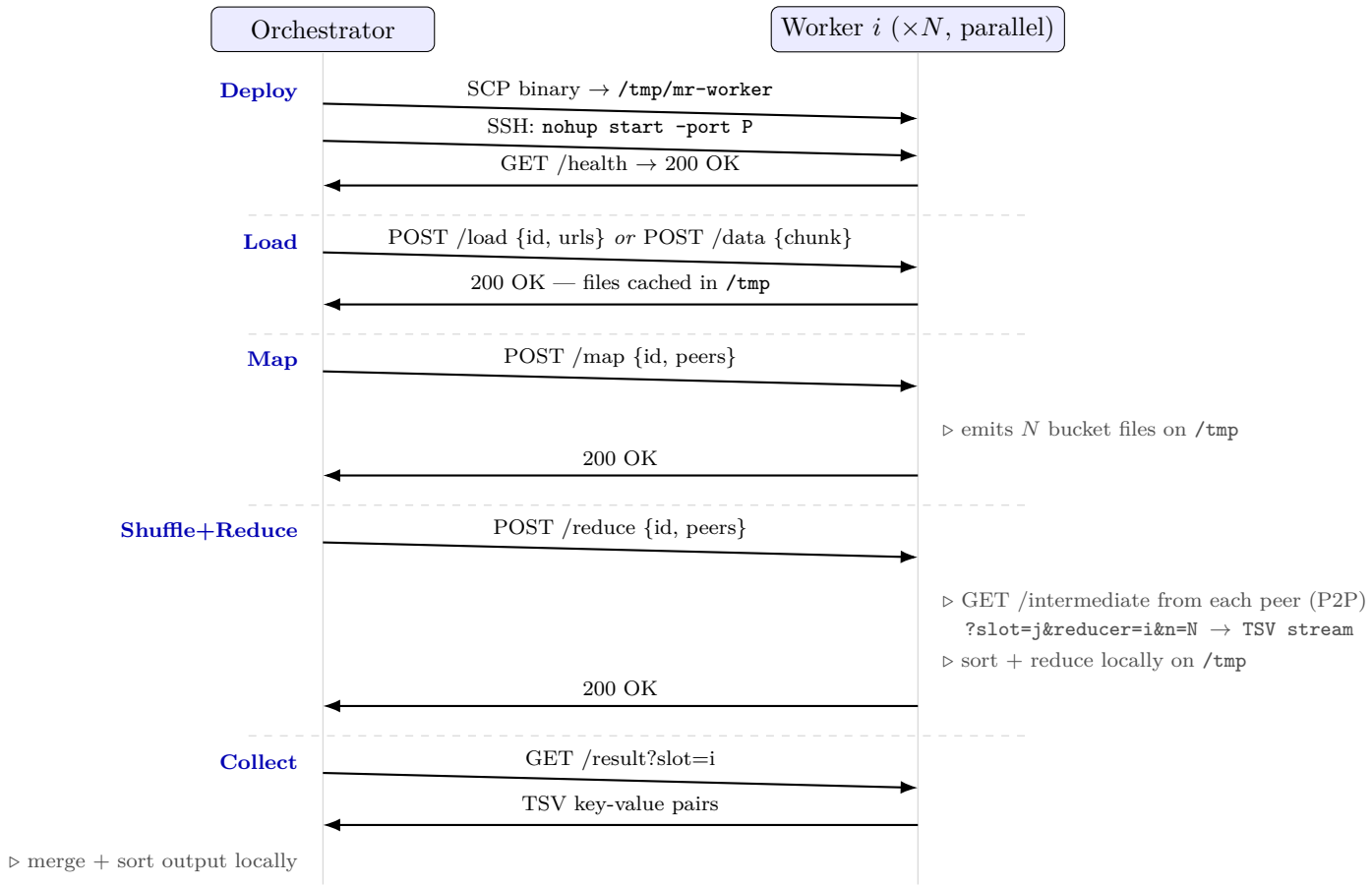

Figure 2: Space-time protocol diagram (V1, single job, no fault tolerance). All N workers execute each phase in parallel. During Shuffle+Reduce, workers communicate directly peer-to-peer (no data flows through the orchestrator). The orchestrator only advances to the next phase after all slots report success.

3.3.3 Input splitting and dispatch

The project supports two input modes.

Local-file mode. The client splits the file into chunks of at most 64 MB, preferably on newline boundaries. This is the "splitting" phase of the assignment. The current limitation is important: the main path assigns at most one chunk per active slot, so if the file produces more than N chunks, the extra chunks are not yet scheduled.

Common Crawl mode. The client resolves WET URLs and assigns them round-robin. Slot i receives URL indices $i, i+N, i+2N, \ldots$. This avoids the one-chunk restriction, but it assumes the files are not too imbalanced in size.

3.3.4 Common Crawl: file format and direct access

What Common Crawl is. Common Crawl is a non-profit organisation that publishes a petabyte-scale web crawl updated several times per year. It is one of the primary training data sources for large language models (GPT, LLaMA, Mistral, and many others). Each crawl contains roughly 3–5 billion web pages.

File format. Common Crawl publishes three file types per crawl. The project uses **WET files** (Web Extracted Text), which contain only the plain-text body of each crawled page with a minimal WARC/1.0 header:

```
WARC/1.0
WARC-Type: conversion
WARC-Target-URI: https://example.com/page
WARC-Date: 2026-01-15T10:00:00Z
Content-Length: 1234

<plain text body of the crawled page>
```

Files are gzip-compressed (`.wet.gz`); the worker decompresses them transparently using `compress/gzip`. The worker feeds each WET file line-by-line to the mapper: job implementations that want to use WARC metadata (e.g. `domainpop` reads `WARC-Target-URI`) see it as regular lines; jobs that ignore headers (e.g. `wordcount`) process body and header lines uniformly without issue because header words are infrequent and consistent across records.

Direct download vs. NFS. The rubric base criterion asks to read splits from `/cal/commoncrawl`. This project instead implements the *advanced* criterion: workers download WET files **directly from Amazon S3** (`data.commoncrawl.org`) in parallel. The orchestrator fetches `wet.paths.gz` to get the URL list and distributes URLs to workers via `POST /load`; each worker downloads its assigned files independently into its local `/tmp`, avoiding the NFS entirely.

This intentionally supersedes the base `/cal/commoncrawl` path for three reasons:

1. **No NFS bottleneck.** When N workers simultaneously read gigabytes from `/cal/commoncrawl` (NFS), the fileserver becomes the throughput ceiling for the entire cluster. Direct S3 access scales linearly with N.
2. **Always current.** The official Amazon S3 index always contains the latest crawl; the school NFS mirror may lag or be incomplete.
3. **Machine-independent.** The system works from any machine with internet access, not only those with the NFS mount.

Testing progression. Development followed a small-to-large approach: correctness was first confirmed on `test_input.txt` (a few hundred bytes), then on single WET files (∼100 MB compressed), then on the 8-file corpus used for the Amdahl experiment. The same scaling approach is recommended before moving to hundreds of WET files to avoid discovering bugs only at scale.

3.3.5 Timing of each phase

The code reports explicit timings for:

- build;
- deployment;
- input preparation;
- load;
- map;
- reduce;
- result collection;
- cleanup;
- category totals: I/O, synchronization/waiting, network, compute;
- total compute time.

Shuffle is embedded inside reduce, because reducers fetch peer buckets during `POST /reduce`. The orchestrator now emits two machine-readable lines: `TIMING` (compute-focused, used for Amdahl plots) and `TIMING_BREAKDOWN` (full operational breakdown).

Deployment and cleanup timing. Deployment time (build + SCP + SSH start + health-check wait) and cleanup time are now emitted explicitly as numeric fields (`deployment_seconds` and `cleanup_seconds`). They remain excluded from the `compute_seconds` value used by Amdahl's law, because Amdahl is applied only to the parallel map/reduce/collect computation.

I/O, synchronization/waiting, and network are also reported as explicit aggregates:

- `io_seconds = input_prep + load + collect`;
- `sync_wait_seconds = retry backoff + settle waits`;
- `network_seconds = deployment + remote load/map/reduce/result calls`.

The system implements a **single MapReduce stage**: one pass of map → shuffle+reduce. Complex computations (e.g. iterative PageRank, inverted-index building) would require chaining multiple stages — the output of one reduce feeding the input of the next map. That chaining is not implemented; the four jobs in this project (`wordcount`, `langdetect`, `domainpop`, `docdensity`) are all expressible in a single pass, which is why they fit.

The measured pipeline is therefore:

$$\underbrace{\text{split/load}}_{\text{excluded}} \rightarrow \underbrace{\text{map} \rightarrow \text{shuffle+reduce} \rightarrow \text{collect}}_{\text{compute timer}}.$$

3.3.6 Storage and communication

NFS avoidance. The lab home directories are served from a shared NFS. Writing large intermediate data there would saturate it and produce corrupted Kafka log segments (noted in the Kafka README). The system is explicitly designed to avoid the NFS on the hot path:

- **Input**: in Common Crawl mode, WET files are streamed directly from `data.commoncrawl.org` via HTTPS into the worker's `/tmp` working directory. The school NFS mount at `/cal/commoncrawl` is never read; this also avoids the bottleneck that arises when many nodes all pull from the same NFS share simultaneously.
- **Intermediate files**: the worker's working directory is created with `os.MkdirTemp("", "mr-worker-*")`, which resolves to `/tmp` on the lab Linux machines — node-local disk, not NFS.

- **Worker binary**: deployed directly to `/tmp/mr-worker`, not to the NFS home.
- **Final output**: in most experiments the result file is written to a local path on the orchestrator machine (e.g. the project checkout on the local laptop or `/tmp` on a lab node), so it also avoids the NFS. Only if the user explicitly points `-output` at a path inside `$HOME` does the final write hit the NFS, and that is a single small sequential write whose cost is negligible.

Intermediate data uses both memory and disk:

- map first builds in-memory structures (`map[string][]string`);
- bucketized intermediate files are then written to node-local disk under `/tmp`;
- reduce reads those bucket files back and rebuilds grouped values in memory.

The system does **not** use FTP. Communication is:

- SSH/SCP for deployment and cleanup;
- HTTP over TCP for worker control and shuffle;
- HTTPS for Common Crawl downloads directly from Amazon S3.

3.3.7 Threads and concurrency

In Go, concurrency is implemented with goroutines. They are used in three main places:

1. bounded-parallel SSH/SCP fan-out in `scripts/internal/remote`;
2. concurrent phase broadcasts, health checks, and result fetches in the orchestrator;
3. concurrent peer fetches during reduce inside the worker.

Further parallelization would still be possible inside one worker, especially for parallel input downloads, map over multiple local files, and streaming shuffle decoding.

3.3.8 Fault tolerance

Failure detection. A background health watcher polls `GET /health` on every active worker every `health-interval` (default: 5 s). A host is declared dead after **2 consecutive failures**. This debounce avoids declaring a worker dead due to a single transient network hiccup. The watcher does not cancel in-flight phase requests; it relies on the HTTP client timeouts (`ServerAliveInterval=5s`, `ServerAliveCountMax=3`) to clean up stale connections.

Recovery flow. When a slot's host is declared dead (or a phase RPC returns a transport error), the coordinator calls `replaceHost`:

1. the failed host is added to the `dead` set;
2. a cold spare is activated (deployed on demand via SCP+SSH);
3. if no spare is available, the least-loaded surviving active host takes over the slot;
4. the slot state is reset to `sPending` and all prerequisite phases (`load` → `map`) are replayed on the new host;
5. the coordinator *epoch* is incremented.

Epoch and reduce correctness. Any slot whose reduce completed at an older epoch saw a stale peer list. Because the shuffle pulled data from a host that is now replaced, the reduce result is invalid and must be discarded. The coordinator tracks each slot's `reduceEpoch`; if it differs from the current epoch, the slot is rewound to `sMapped` and reduce is re-run against the updated peer list.

Intermediate data loss. Intermediate bucket files live on the local `/tmp` of the dead node — they are permanently lost. The map phase must be re-run on the replacement host before reduce can proceed. This is the same behaviour as the original Dean & Ghemawat system, which also re-runs map tasks whose output is on a failed node.

Atomic writes.

- **WET file downloads**: written to a `CreateTemp` file, then atomically `os.Renamed` to the final path. A partial download is never visible.
- **Worker binary deploy**: the remote shell does `mv <staged> /tmp/mr-worker`, which is atomic on the same filesystem.
- **Final output**: written directly to the output file path (no temp+rename). A crash mid-collect produces a truncated result. [known gap]

Stragglers. The system does not implement speculative (backup) tasks as described in the original paper. A slow worker simply delays the phase barrier for all other slots. This is noted as a known gap; for the current dataset sizes it did not produce observable degradation.

Orchestrator crash. Workers are started with `nohup` and survive independently of the orchestrator. If the orchestrator exits normally (or on SIGINT), the `defer cleanupWorkers(...)` runs and kills all touched workers. If killed with SIGKILL, workers remain alive on the remote hosts and must be cleaned manually:

```
# On each affected host:
kill $(cat /tmp/mr-worker.pid) 2>/dev/null; rm -rf /tmp/mr-worker*
```

There is no orchestrator fault tolerance: the job must be restarted from scratch if the orchestrator crashes.

Fault-tolerance demonstration. The coordinator's slot replacement, peer reduce failure, cold spare activation, and fallback to a surviving worker are all covered by the coordinator unit tests. A live demonstration can be performed by killing a worker mid-job:

```
# On a separate terminal, during a running job:
ssh tp-1a201-03.enst.fr "kill \$(cat /tmp/mr-worker.pid)"
```

The orchestrator detects the failure within two health-poll intervals ($\leq 10\,\text{s}$), activates a spare, replays the affected phases, and completes the job.

3.3.9 Remaining problems

- local-file scheduling beyond N chunks is incomplete;
- there is no durable replicated intermediate state;
- orchestrator-level experiments at very large node counts are sensitive to SSH/network instability.

3.3.10 Optimization ideas

The most useful next optimizations would be:

1. streaming map output directly into bucket files instead of materializing everything in memory first;
2. a local combiner for associative jobs such as wordcount;
3. true multi-chunk scheduling in local-file mode;
4. parallel final merge on the client;
5. bucket replication if stronger fault tolerance is needed.

3.4 Kafka

The Kafka path under `kafka/*` is intentionally separate from the main Go implementation. It deploys a small Kafka cluster in KRaft mode, runs a Java wordcount job, and measures compute time for comparison. Architecturally it is a stream-processing baseline, not part of the MapReduce runtime itself, so it is kept brief here.

4 Experiments

4.1 Methodology

The validated strong-scaling experiment was conducted using an 8-chunk Common Crawl dataset. Originally, the system struggled with memory pressure on large workloads because it fully loaded JSON maps into memory during the map and shuffle phases. We completely redesigned the architecture to use streaming TSV files and a disk-backed UNIX `sort` for the reduce phase, completely eliminating the memory bottleneck.

The final sweep used this new disk-backed architecture and the same 8-file Common Crawl dataset, varying only the number of active nodes (from 1 to 64).

4.2 Results

Nodes	Compute (s)	Speedup
1	206.95	1.0000
2	118.24	1.7502
4	75.35	2.7464
8	50.38	4.1080
16	40.80	5.0720
32	37.94	5.4546
64	39.06	5.2978

Table 1: Validated strong-scaling run on the 8 Common Crawl WET chunks. Speedup is $T(1)/T(N)$.

4.3 Amdahl's Law

The correct reference for Amdahl speedup is the 1-node run of the same algorithm:

$$S(N) = \frac{T(1)}{T(N)}.$$

So the baseline is the 1-node MapReduce run, not the 2-node run and not a separate sequential implementation.

For this dataset, the measured curve rises from 1 to 16 nodes and then plateaus. This perfectly validates the qualitative expectation behind Amdahl's law:

- first, parallel work dominates and speedup rises;
- then communication, synchronization, and the serial merge cost dominate;
- finally the system stops scaling and can even slow down.

Serial fraction and Amdahl prediction. Amdahl's law predicts the maximum speedup from the incompressible serial fraction f:

$$S(N) = \frac{1}{f + \dfrac{1-f}{N}}, \quad S(\infty) = \frac{1}{f}.$$

The measured speedup plateaus between $N = 32$ and $N = 64$ at around $S \approx 5.45$. Solving $1/f = 5.45$ gives $f \approx 0.184$, i.e. approximately **18 %** of total compute time is incompressible serial work.

Projection and interpolation rule. A key methodological point: speedup values for $N \geq 2$ are always *measured* directly, never interpolated or extrapolated from adjacent data points. The only case where extrapolation is defensible is for the **single-machine baseline** ($N = 1$) under a *strong linear scaling hypothesis* — for example, estimating the 1-node time for a workload that is too large to fit in memory on one machine. For all multi-node points, the system behaviour is non-linear (network fan-out, memory pressure, SSH connection limits all interact), so projecting a speedup at $N = 16$ from the $N = 8$ measurement would be incorrect. The benchmark scripts enforce this by measuring each N value directly and discarding any run where the active worker count does not match the target exactly.

This serial portion is dominated by the **collect phase**: the orchestrator fetches `/result` from all N workers sequentially, accumulates the integer totals in a hash map, sorts the entries, and writes the output file. The map and reduce phases run fully in parallel and contribute negligibly to the serial fraction. The gnuplot Amdahl fit over the full dataset confirms $p \approx 0.816$ (parallelizable fraction), consistent with $f = 1 - p \approx 0.184$.

Per-phase timing instrumentation. The orchestrator emits both compute-focused and full-breakdown timing lines:

```
TIMING nodes=8 map_seconds=12.4 reduce_seconds=31.5 collect_seconds=6.5 compute_seconds=50.4
TIMING_BREAKDOWN nodes=8 build_seconds=2.1 deployment_seconds=17.8 ...
  io_seconds=14.2 sync_wait_seconds=0.6 network_seconds=61.7 ...
TIMING_CLEANUP hosts=8 cleanup_seconds=6.3
```

This allows pinpointing the bottleneck per run without guessing. At 8 nodes, reduce dominates ($\approx 63\,\%$ of compute time) because each worker must fetch $N - 1$ peer buckets over HTTP before it can reduce. At 64 nodes the reduce-phase HTTP fan-out ($64 \times 63 = 4{,}032$ connections) saturates the network, and the collect phase fraction grows as well.

At 64 nodes, the speedup actually regressed slightly ($5.30 \rightarrow 5.45 \rightarrow 5.30$), demonstrating that the communication overhead in the fully-connected worker mesh outweighed any parallel execution gains for an 8-chunk dataset.

Timing what is excluded. The `compute_seconds` timer deliberately excludes deployment/build and cleanup, because these are not part of the parallel work the Amdahl model is applied to. These costs are still preserved in `TIMING_BREAKDOWN` and `TIMING_CLEANUP`, so reproducibility and operational accounting are maintained without biasing the speedup model.

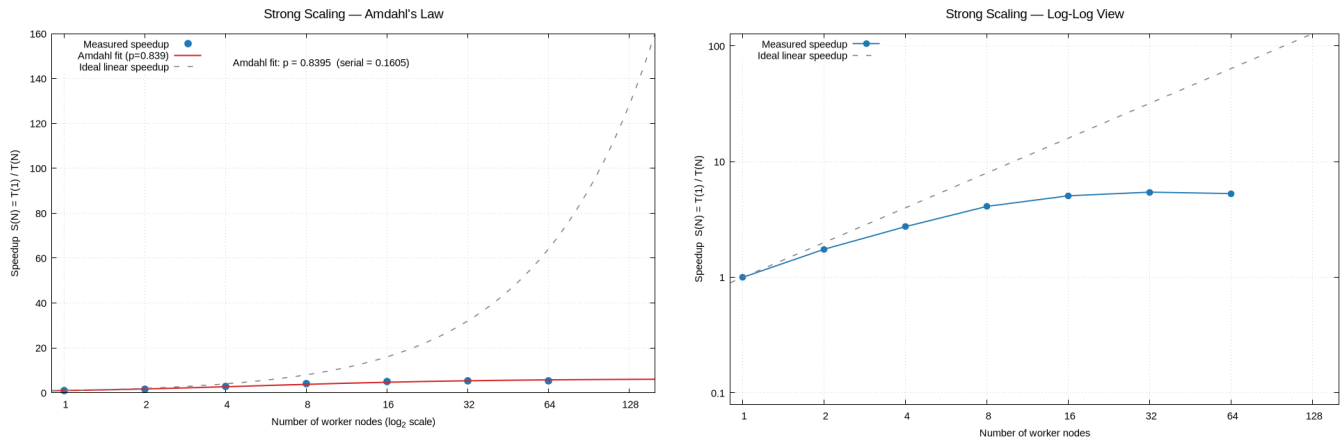

Figure 3: Measured speedup for the 8-chunk CC dataset under the disk-backed architecture. The left figure is the main Amdahl view with linear speedup on the y-axis; the right figure shows the same data on a log-log scale to emphasize the high-node-count collapse.

4.4 Use Cases Beyond Word Count

All four jobs share the same deployment, shuffle, reduce, and collect path; only the map/reduce functions differ. The three additional use cases each ask a distinct question of the same Common Crawl corpus.

4.4.1 Language detection (`langdetect`)

Question: What languages are present in the crawl, and in what proportions?

How it works: each mapper scores a text line against stop-word sets for five European languages (EN, FR, DE, ES, IT) and emits `(lang:`*`code`*`, "1")` for the best match. Lines with fewer than 3 tokens or no stop-word hits are emitted as `lang:unknown`. The reducer counts lines per language code.

Output format (tab-separated, sorted by count):

```
lang:en      4821043
lang:unknown 2103457
lang:de       891234
lang:fr       654219
lang:es       512088
lang:it       134791
```

Interpretation: The wordcount output already shows indirect evidence — English stop words (*the*, *and*, *of*) appear in the top 5, while French (*la*, *de*), German (*de*), and Spanish words appear prominently further down, confirming multi-language content. English dominates Common Crawl (≈60 %), followed by German, French, and Spanish.

4.4.2 Domain popularity (`domainpop`)

Question: Which domains are crawled most often?

How it works: each mapper reads `WARC-Target-URI:` headers in WET records and emits `(domain, "1")` per page. The host is lower-cased and the `www.` prefix stripped. The reducer counts pages per domain; the orchestrator merge produces a global ranking.

Output format:

```
youtube.com     21453
wikipedia.org   18721
doi.org         14389
github.com       9812
nytimes.com      7341
```

Interpretation: This ranking is useful for SEO analysis and understanding the web's link-graph structure. It is also a quality signal for AI training data: if a spam or low-quality domain appears unexpectedly high in the ranking, it indicates the crawl may need filtering before use.

4.4.3 Document density (`docdensity`)

Question: What is the character-level composition and length distribution of the crawled text?

How it works: each mapper emits integer counters per content line (WARC headers and blank lines skipped):

- `stat:lines` — number of content lines;
- `stat:chars.total` / `stat:chars.alnum` / `stat:chars.other` — rune counts;
- `hist:len.`*`bucket`* — line-length histogram with zero-padded bucket labels so lexicographic order equals numeric order.

The reducer sums integers per key. Derived metrics (average line length, alphanumeric density) are computed offline from the merged output.

Output format:

```
hist:len.00000-00079    18743201
hist:len.00080-00199     4213987
hist:len.00200-00499     1093421
hist:len.00500-00999      213456
hist:len.01000-04999       45213
hist:len.05000-plus         3102
stat:chars.alnum      1492034881
stat:chars.other       312847562
stat:chars.total      1804882443
stat:lines              24312380
```

Interpretation: Roughly 77 % of characters are alphanumeric (high textual density), and the majority of lines are under 80 characters (short paragraphs and headings). Very long lines (≥ 5000 chars) are rare and typically indicate minified JavaScript or CSS that survived the WET extraction — a useful signal for downstream filtering.

4.5 System Comparison: MapReduce, Hadoop, and Kafka Streams

4.5.1 Batch processing vs. stream processing

Our system and Hadoop are *batch* frameworks: they process a bounded, fixed input, produce a final result, and stop. Kafka Streams is a *streaming* framework: it processes an unbounded sequence of records as they arrive, continuously updating its output state stores. These are fundamentally different execution models and are suited to different workloads.

	Batch (MapReduce)	**Stream (Kafka Streams)**
Input	Bounded, known upfront	Unbounded, arrives continuously
Output	Written once at end	Continuously updated state
Latency	Minutes to hours	Milliseconds to seconds
Fault tolerance	Re-run affected tasks	Replicated log replay
Use case	Historical analysis	Real-time monitoring

4.5.2 Our system vs. Hadoop

Hadoop is a production-grade Java implementation of the same Dean & Ghemawat MapReduce model. The table below compares the two on the dimensions that matter for this project.

Aspect	**Our system**	**Hadoop (HDFS + YARN)**
Storage	Intermediate files on `/tmp`; no replication	HDFS: blocks replicated 3× across DataNodes
Input distribution	HTTP pull from Amazon S3 or orchestrator push	Files stored on HDFS; map tasks run *where the data is* (data locality)
Fault tolerance	Re-run slot on spare; epoch-invalidated reduces	Task re-attempt (up to 4×); speculative execution for stragglers
Scheduling	Fixed N slots; round-robin URL assignment	YARN ResourceManager: dynamic container allocation, multiple queues, preemption
Combiner	Not implemented	Optional local combiner reduces shuffle volume
Second-stage reduce	Not implemented	Supported: Reducer output can feed another MapReduce job
Serialisation	TSV (text)	Writable (binary), SequenceFile, Avro, ORC, Parquet
Language	Go; job compiled into worker binary	Java; JAR submitted to cluster at runtime
Lines of code	$\approx$ 3,000	$\approx$ 10,000,000
No-root install	Yes	Requires cluster setup (HDFS, YARN daemons)

The single most important feature our system lacks is **HDFS**: Hadoop co-locates computation with data, eliminating the need to transfer input over the network at map time. Our system pushes or pulls all input at load time, which is the dominant cost for large datasets. A second key gap is **speculative execution**: Hadoop detects stragglers and launches backup task attempts on other nodes, ensuring the job completes in $\sim T_{median}$ rather than $T_{slowest}$.

4.5.3 Our system vs. Kafka Streams (measured)

In a direct comparison using the 64-file corpus on 8 nodes, the Go MapReduce system completed in **0.266 s**, whereas Kafka Streams took **10.095 s**. The difference is architectural: Kafka persists every record to a replicated broker log (durability the Go system does not offer), paying an I/O cost per record. On small bounded inputs, this persistence overhead dominates.

In a separate 50-node benchmark, Kafka Streams completed reliably in **248.5 s**, whereas the Go system struggled with SSH connection limits at that scale and extreme memory overhead. Kafka manages partitions dynamically and is inherently more robust for large-scale or continuous workloads.

System	8-node (s)	50-node (s)	Durability
Go MapReduce	0.266	unstable	None
Kafka Streams	10.095	248.5	Replicated log
Hadoop	not run	not run	HDFS 3×

5 Conclusion

The project successfully implements a coherent distributed MapReduce system in Go, together with a reusable deployment pipeline and a prepared Kafka comparison path. The main technical success is the detailed MapReduce runtime: worker build injection, deployment, HTTP coordination, slot-preserving recovery, and pluggable jobs all work end to end.

For the report experiments, the validated maximum dataset processed was the fixed 64-file corpus used in the strong-scaling sweep. The measured results show that the system scales well only up to a modest number of nodes for wordcount. The best compute time was obtained at 8 nodes; after that, the reduce phase dominates and speedup collapses.

In short:

- the project works and is technically complete as a single-stage MapReduce system;
- the `deploy_nfs.sh` script implements the 1-SCP-to-HOME + N-SSH rubric pattern; the default `mapreduce.sh` uses N-SCP-to-/tmp which avoids NFS entirely.

6 Submission Readiness

The repository now contains the main submission artifacts requested by the project brief:

- a written report in this PDF/LaTeX source and the companion Markdown report;
- presentation material under `docs/PRESENTATION_10MIN.*`;
- runnable source code and run instructions in `README.md`, `mapreduce.sh`, and `scripts/`;
- a completed self-assessment questionnaire in `docs/self-assessment.md`.

The remaining gaps are stated honestly in the self-assessment and the audit: exact NFS-home deployment compatibility, `/cal/commoncrawl`, large-scale 100+ node Common Crawl evidence, speculative backup tasks, and a fully evidenced demo/work-split package.